

Laporan Tugas Besar
UAS BIG DATA
Prediksi Tarif berdasarkan Jarak Menggunakan Machine Learning



Anggota Kelompok:

Muhammad Fitran Ramadhan	1103220156
Fahrial Al Akbar Warsito Putra	1103220096
Muhammad Darul Rizqullah	1103223203
Muhammad Wira Alfikri	1103223104

Dosen Pengampu: Angel Metanosa Afinda, S.Kom., M.Kom.

Program Studi S1 Teknik Komputer
Fakultas Teknik Elektro
Universitas Telkom
2025

Daftar Isi

Daftar Isi	i
Lampiran: Kontribusi Anggota Kelompok	iii
I. Pendahuluan.....	1
1.1 Latar Belakang	1
1.2 Tujuan Penelitian	1
1.3 Ruang Lingkup.....	1
II. Studi Pustaka	2
2.1 Eksplorasi Data (Exploratory Data Analysis - EDA)	2
2.1.1 PySpark dalam Analisis Data Penjelasan mengenai PySpark sebagai kerangka kerja pemrosesan data skala besar. Bahas konsep <code>SparkSession</code> , <code>DataFrame</code> API, dan keunggulan paralelisme dalam EDA.	2
2.1.2 Agregasi Data.....	2
2.1.3 Pemrosesan Data Streaming	2
2.2 Machine Learning	3
2.3 Pemilihan Algoritma dan Metode Evaluasi	3
2.4 Visualisasi Hasil melalui Dashboard	4
III. Metodologi.....	5
3.1 Alur Umum Penelitian	5
3.2 Deskripsi Dataset	5
3.3 Akuisisi dan Pembacaan Data dengan PySpark.....	6
3.4 Pembersihan dan Transformasi Data	8
3.5 Eksplorasi Data (EDA)	9
3.8 Pembangunan Model Machine Learning	12
3.9 Evaluasi Model	13
3.10 Pembuatan Dashboard Interaktif.....	14
3.11 Pertanyaan Bisnis yang Dieksplorasi	14
IV. Hasil dan Pembahasan	15
4.1 Hasil Eksplorasi Data.....	15
4.2 Hasil Agregasi Data	19
4.3 Hasil Analisis Data Streaming	21
4.4 Hasil Evaluasi Model Machine Learning	22
4.5 Pembahasan Model dan Interpretasi	23
4.6 Insight dari Pertanyaan Bisnis	24

4.7 Action Plan Bisnis.....	24
4.8 Visualisasi Dashboard Interaktif.....	25
V. Kesimpulan dan Saran	27
5.1 Kesimpulan.....	27
5.2 Saran	27
VI. Daftar Pustaka.....	27
- https://gemini.google.com/	27
VII. Lampiran	27
7.1 Kode Program	27
7.2 Visualisasi dan Tabel	28

Lampiran: Kontribusi Anggota Kelompok**Judul Tugas Besar:** Prediksi Tarif berdasarkan Jarak Menggunakan Machine Learning**Nama Kelompok:** Kelompok pertama

NIM	Nama Lengkap	Peran/Kontribusi Utama	Persentase Kontribusi (%)
1103220156	Muhammad Fitran Ramadhan	Data Streaming	25%
1103220096	Fahrial Al Akbar Warsito Putra	Download Data Set, Import, Pembersihan data, EDA	25%
1103223203	Muhammad Darul Rizqullah	Model Machine Learning	25%
1103223104	Muhammad Wira Alfikri	Visualisasi Dashboard	25%

I. Pendahuluan

1.1 Latar Belakang

Dalam era digital saat ini, data telah menjadi aset yang sangat berharga. Jumlah data yang sangat besar ini perlu diolah dan dianalisis untuk mendukung pengambilan keputusan yang tepat dan berdasarkan fakta. Salah satu pendekatan utama dalam pengolahan data besar adalah dengan menggunakan *Exploratory Data Analysis* (EDA) dan teknik *Machine Learning*. EDA merupakan tahap awal yang krusial dalam proses analisis data, di mana kita mengeksplorasi struktur dan pola dalam data menggunakan statistik deskriptif dan visualisasi. Langkah ini membantu dalam memahami karakteristik data secara lebih mendalam, mendeteksi anomali, serta membentuk hipotesis awal sebelum melangkah ke tahap pembangunan model *machine learning*.

Dalam tugas besar ini, kami menerapkan EDA dan *machine learning* pada dataset NYC Yellow Taxi Trip Data. Dataset ini akan dianalisis untuk menggali pola-pola perjalanan, memahami faktor-faktor yang memengaruhi durasi perjalanan, serta membangun model prediktif. Hasil analisis ini ditujukan untuk mengembangkan sebuah model yang dapat mendukung pengambilan keputusan berbasis data (*data-driven decision making*) di sektor transportasi urban.

1.2 Tujuan Penelitian

Tujuan utama dari penelitian ini adalah sebagai berikut:

- Melakukan eksplorasi pada dataset NYC Yellow Taxi untuk mengidentifikasi pola-pola yang ada.
- Membangun model *machine learning* untuk memprediksi durasi perjalanan taksi.
- Mengevaluasi kinerja model *machine learning* yang telah dibangun.

1.3 Ruang Lingkup

Penelitian ini memiliki batasan sebagai berikut:

- Data: Data yang digunakan adalah data trip taksi kuning NYC yang bersifat campuran (numerik dan kategorikal).
- Algoritma: Algoritma *machine learning* yang akan diterapkan adalah Regresi Linier untuk prediksi durasi perjalanan.
- Asumsi: Diasumsikan bahwa data yang digunakan telah melalui proses pembersihan awal, meskipun akan tetap dilakukan penanganan terhadap *missing values* dan *outliers*.

II. Studi Pustaka

2.1 Eksplorasi Data (Exploratory Data Analysis - EDA)

EDA adalah proses investigasi awal pada data untuk menemukan pola, anomali, menguji hipotesis, dan memeriksa asumsi dengan bantuan statistik deskriptif dan representasi grafis. Tujuannya adalah untuk memahami struktur data, mengidentifikasi *outliers*, distribusi variabel, dan korelasi antar fitur. Teknik yang digunakan meliputi statistik deskriptif dan visualisasi data seperti histogram, *boxplot*, dan *scatter plot*.

2.1.1 PySpark dalam Analisis Data

Penjelasan mengenai PySpark sebagai kerangka kerja pemrosesan data skala besar. Bahas konsep *SparkSession*, *DataFrame* API, dan keunggulan paralelisme dalam EDA.

PySpark adalah *framework* pemrosesan data skala besar yang menyediakan API Python untuk Spark. Konsep utama dalam PySpark meliputi *SparkSession* sebagai titik masuk ke fungsionalitas Spark, dan *DataFrame* API yang memungkinkan pemrosesan data terdistribusi secara paralel.

2.1.2 Agregasi Data

Agregasi data adalah meringkas data berdasarkan kolom dan menjadikan 1 kolom sebagai acuan. Agregasi disini menggunakan teknik `groupBy()`, `agg()`, dan fungsi agregasi yang digunakan adalah `sum()`, `avg()`, dan `count()`. kami melakukan 3 agregasi terhadap 3 kolom, yaitu `PULocationID`, `Pickup_dayofweek`, dan `rate_code_id`.

2.1.3 Pemrosesan Data Streaming

Data streaming adalah data yang terus-menerus dihasilkan oleh berbagai sumber. Data tersebut harus diproses secara bertahap menggunakan teknik pemrosesan aliran tanpa memiliki akses ke semua data. Proses yang pertama dilakukan adalah inisialisasi `sparksession` dengan `environ` path `JAVA_HOME` dan `SPARK_HOME`. Proses kedua adalah membuat folder streaming dari input, output, dan checkpoint dalam folder content. Proses ketiga menyiapkan skema dataset dengan membuat masing-masing kolom. setelah semua siap, lakukan pembacaan streaming CSV dari spark dan lakukan pemanggilan dengan `input_path` dan melakukan penyimpanan ke dataset `df_stream` kedalam `out_path`. Lakukan pengiriman data simulasi ke folder input menggunakan 2 databases, yaitu database yang sudah dilakukan clean, dan databases yang sudah di spark.

2.2 Machine Learning.

Machine Learning (ML) adalah cabang dari kecerdasan buatan (AI) yang memungkinkan komputer belajar dari data dan membuat prediksi atau keputusan tanpa diprogram secara eksplisit. Terdapat beberapa kategori algoritma ML, antara lain:

- **Supervised Learning:** Algoritma belajar dari data berlabel, seperti klasifikasi dan regresi. Contoh: Random Forest, Linear Regression, XGBoost, Gradient Boosting.
- **Unsupervised Learning:** Algoritma belajar dari data tanpa label untuk menemukan struktur tersembunyi. Contoh: Clustering (K-Means).
- **Reinforcement Learning:** Model belajar dari interaksi dengan lingkungan melalui sistem reward dan punishment.

Metode algoritma Machine Learning yang kelompok kami pilih ini berbasis Supervised Learning untuk memprediksi tarif berdasarkan jarak dengan menggunakan model, yaitu Gradient Boosted Tree Regressor dan Linear Regression.

2.3 Pemilihan Algoritma dan Metode Evaluasi

Pemilihan algoritma merupakan hal yang penting yang bergantung pada karakteristik data. Karakteristik data pada dataset yang kami gunakan yaitu berupa numerik dengan contohnya seperti `passenger_count` dan `fare_amount`. Berdasarkan karakteristik tersebut, algoritma yang dipilih harus mampu menangani karakteristik data tersebut yaitu menggunakan algoritma regressor dengan modelnya yaitu:

- a. **Gradient Boosted Tree Regressor**, model ini merupakan salah satu algoritma *ensemble* yang kuat. GBT membangun model secara bertahap dalam bentuk pohon keputusan (*decision trees*), di mana setiap pohon baru mencoba memperbaiki kesalahan dari pohon sebelumnya. Model ini sangat efektif dalam menangani data tabular dengan fitur campuran (numerik dan kategorikal) dan mampu menangkap pola non-linier yang kompleks, yang seringkali ada dalam data dunia nyata seperti data transportasi.
- b. **Linear Regression**, model ini dipilih sebagai *baseline* karena kesederhanaannya dan kemampuannya untuk menginterpretasikan hubungan linier antara variabel independen dan dependen. Regresi Linear mengasumsikan bahwa ada hubungan garis lurus antara fitur input dan variabel output.

Metode evaluasi yang digunakan, yaitu:

- a. **Root Mean Squared Error (RMSE):** RMSE mengukur rata-rata besarnya kesalahan prediksi model. Metrik ini menghitung akar kuadrat dari rata-rata selisih kuadrat antara

nilai aktual dan nilai prediksi. Nilai RMSE yang lebih rendah menunjukkan performa model yang lebih baik, karena artinya selisih antara prediksi dan kenyataan lebih kecil.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y})^2}$$

dimana n adalah jumlah data, y_i adalah nilai aktual, dan $\hat{y}$ adalah nilai prediksi.

- b. **Koefisien Determinasi (R-squared atau R^2):** R^2 mengukur seberapa baik model regresi mampu menjelaskan variasi dalam data. Nilai R^2 berkisar antara 0 dan 1, di mana nilai yang lebih mendekati 1 menunjukkan bahwa model mampu menjelaskan sebagian besar variabilitas data target.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y})^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

dimana n adalah jumlah data, y_i adalah nilai aktual, dan $\hat{y}$ adalah nilai prediksi serta $\bar{y}$ adalah rata-rata dari nilai aktual.

2.4 Visualisasi Hasil melalui Dashboard

Penyampaian hasil analisis data dan performa model kepada orang penting seringkali tidak efektif jika hanya disajikan dalam bentuk tabel atau laporan statis. Oleh karena itu, visualisasi melalui *dashboard* interaktif menjadi sangat penting.

Dashboard berfungsi sebagai jembatan antara analisis teknis dan pengambilan keputusan bisnis. Manfaat utamanya adalah:

- **Tampilan yang mudah dicerna**, mengubah data dan angka kompleks menjadi visualisasi yang mudah dipahami seperti grafik, peta, dan bagan.
- **Interaktif**, memungkinkan pengguna untuk berinteraksi dengan data, misalnya dengan memfilter berdasarkan waktu, lokasi, atau kategori tertentu untuk mendapatkan wawasan yang lebih dalam.

Dalam proyek ini, kami akan membuat dashboard dengan menggunakan aplikasi Microsoft Power BI dengan fitur utama:

1. **Hasil Analisis Eksplorasi Data (EDA):** Visualisasi distribusi data, korelasi antar fitur, dan statistik deskriptif.
2. **Performa Model:** Menampilkan metrik evaluasi (RMSE dan R^2) untuk kedua model secara perbandingan.

III. Metodologi

3.1 Alur Umum Penelitian

Penelitian ini mengikuti alur kerja yang sistematis untuk memastikan proses analisis data dan pembangunan model berjalan dengan terstruktur. Tahapan penelitian digambarkan sebagai berikut:

1. **Akuisisi Data:** Mengunduh dataset NYC Yellow Taxi dari website kaggle.
2. **Inisialisasi Lingkungan PySpark:** Menyiapkan SparkSession sebagai titik masuk untuk pemrosesan data terdistribusi.
3. **Eksplorasi Data Awal (EDA):** Memuat data ke dalam PySpark DataFrame dan melakukan analisis awal untuk memahami struktur, tipe data, distribusi, dan mengidentifikasi nilai yang hilang atau anomali.
4. **Pembersihan dan Transformasi Data:**
 - Menangani nilai yang hilang (*missing values*).
 - Menerapkan Min dan Max untuk Standarisasi pada data
 - Menampilkan data sebelum dan sesudah pembersihan dan transformasi data
5. **Pembangunan Data Streaming:** Pembangunan data streaming dari data yang sudah dibersihkan dan ditransformasi data.
6. **Pembangunan Pipeline Machine Learning:**
 - Mendefinisikan model Regresi Linear dan GBT Regressor.
 - Membangun Pipeline di PySpark yang menggabungkan semua tahap transformasi data dan pemodelan.
 - Menampilkan hasil grafik dari masing-masing algoritma Machine Learning.
7. **Pelatihan dan Evaluasi Model:**
 - Melatih kedua *pipeline* pada data pelatihan.
 - Membuat prediksi pada data pengujian.
 - Mengevaluasi performa model menggunakan metrik RMSE dan R^2 .
8. **Pembuatan Dashboard:** Merancang dan membangun *dashboard* interaktif menggunakan aplikasi Microsoft Power BI untuk memvisualisasikan hasil EDA, perbandingan model.

3.2 Deskripsi Dataset

Asal-usul dataset berasal dari website [kaggle.com](https://www.kaggle.com) dengan judul “NYC Yellow Taxi Trip Data” yang berisi catatan perjalanan taksi kuning di New York City. Struktur dari dataset

tersebut yaitu terdiri dari 16 kolom dan jumlah baris 8217552. Tipe data pada dataset yaitu berupa numerik. Berikut adalah fitur-fitur utama:

- vendorrp
- tpep_pickup_datetime
- tpep_dropoff_datetime
- passenger_count
- trip_distance
- RatecodeID
- store_and_fwd_flag
- PuLocationID
- DoLocationID
- payment_type
- fare_amount
- extra
- mta_tax
- tip_amount
- tolls_amount
- improvement_surcharge
- total_amount
- congestion_surcharge
- orderID

Tujuan kami mengambil dataset ini, untuk memprediksi tarif berdasarkan jarak.

3.3 Akuisisi dan Pembacaan Data dengan PySpark

Langkah fundamental dalam setiap aplikasi PySpark adalah inisialisasi SparkSession. Objek SparkSession berfungsi sebagai titik masuk tunggal (single point of entry) untuk berinteraksi dengan fungsionalitas Spark dan sebagai pengelola sumber daya klaster. Inisialisasi dilakukan dengan menggunakan pola desain builder, yang memastikan bahwa hanya satu sesi yang aktif untuk setiap aplikasi, atau membuat sesi baru jika belum ada.

```

from pyspark.sql import SparkSession
from pyspark.sql.types import StructType, StructField, StringType, IntegerType

spark = SparkSession.builder \
    .appName("Streaming_NYC_Taxi") \
    .master("local[*]") \
    .getOrCreate()

```

Parameter `.appName()` digunakan untuk memberikan nama yang dapat diidentifikasi pada aplikasi, yang berguna untuk pemantauan pada antarmuka pengguna (UI) Spark. Perintah `.getOrCreate()` akan mengambil sesi yang ada atau membuat yang baru, sehingga menjamin efisiensi sumber daya.

Setelah lingkungan Spark siap, proses akuisisi data dilakukan dengan membaca file dari sistem penyimpanan. Hasil dari operasi pembacaan ini adalah sebuah DataFrame PySpark, yaitu struktur data terdistribusi yang diabstraksikan dalam bentuk tabel dengan kolom-kolom bernama. Antarmuka `spark.read` menyediakan fungsionalitas untuk menangani berbagai format data.

```

from pyspark.sql.functions import count, when, isnull
import os

combined_output_path = "/content/combined_streaming_output"

df_combined = spark.read.csv(combined_output_path, header=True, inferSchema=True)

# Mengecek jumlah nilai null di setiap kolom
df_combined.select([count(when(isnull(c), c)).alias(c) for c in df_combined.columns]).show()
df_combined.describe().show()

```

File CSV (Comma-Separated Values) merupakan format yang umum digunakan. Untuk memuatnya secara benar, diperlukan konfigurasi parameter spesifik. Parameter `header=True` menginstruksikan Spark untuk menggunakan baris pertama dari file sebagai header kolom DataFrame. Sementara itu, `inferSchema=True` mengaktifkan mekanisme inferensi skema otomatis, di mana Spark menganalisis sebagian data untuk menentukan tipe data yang paling sesuai untuk setiap kolom (misalnya, Integer, Double, String), sehingga menghindari perlakuan semua data sebagai teks.

4.4 Baca Streaming CSV

```

[ ] # Baca file streaming
df_stream = spark.readstream \
    .format("csv") \
    .option("header", "true") \
    .schema(schema) \
    .load(input_path)

```

Untuk kasus penggunaan data real-time, di mana data datang secara kontinu, PySpark menyediakan antarmuka `spark.readStream`. Metode ini menghasilkan `DataFrame` yang akan diperbarui secara dinamis seiring dengan masuknya data baru. Dalam pemrosesan stream, pendefinisian skema secara manual sangat direkomendasikan untuk memastikan konsistensi dan stabilitas.

Proses akuisisi data dalam PySpark secara inheren menghasilkan objek `DataFrame`. Operasi `spark.read` (untuk data batch) dan `spark.readStream` (untuk data streaming) adalah metode langsung untuk memuat data sekaligus menstrukturkannya. Dengan demikian, tidak ada langkah "konversi" terpisah yang diperlukan; objek yang dihasilkan dari operasi baca sudah menjadi `DataFrame` yang siap untuk tahap transformasi dan analisis selanjutnya.

3.4 Pembersihan dan Transformasi Data

Pertama-tama kita mengecek nilai Null atau missing values dari dataset yang telah diambil. Berikut adalah hasil dari pengecekan missing values:

VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type
50717			50717		50717	50717			50717

fare_amount	extra	mta_tax	tip_amount	tolls_amount	improvement_surcharge	total_amount	congestion_surcharge
							4855978

Disini dapat terlihat nilai missing values pada tiap kolom. Setelah mengetahui nilai missing value, kita bersihkan nilai nulnya dengan menghapus baris yang memiliki nilai null pada `VendorID`, `Passenger_count`, `RatecodeID`, dll. Sedangkan untuk kolom `congestion_surcharge` karena memiliki banyak sekali nilai null sebesar 485598. Oleh karena itu untuk kolom `congestion_surcharge` dihapus agar data tidak banyak yang hilang. Berikut adalah jumlah baris sebelum dan sesudah pembersihan:

```
[12] # Menghitung jumlah baris sebelum dan setelah pembersihan
      print("Jumlah baris sebelum pembersihan:", df_taxi.count())
      print("Jumlah baris setelah pembersihan:", df_taxi_clean.count())
```

➡ Jumlah baris sebelum pembersihan: 8217552
 Jumlah baris setelah pembersihan: 7958541

Kemudian kita tampilkan `data.describe` yang sudah diclean untuk melihat ringkasan data seperti count, mean, sttdev, min, max pada tiap kolom.

summary	VendorID	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type	fare_amount
count	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835
mean	1.6344272903762596	1.5541891875306407	2.804965205002796	1.057689545582836	NULL	165.17685737498864	163.24591999226138	1.2967593933267907	12.374865329182482
stddev	0.5370005244944361	1.2138082405879083	8.726846318802948	0.7106955407481079	NULL	66.55563934993339	70.5859165149234	0.47748899777085263	253.95456381530767
min	1	0	0.0	1	N	1	1	1	-362.0
max	4	9	22543.99	99	Y	265	265	5	623259.86

RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type	fare_amount	extra	mta_tax	tip_amount	tolls_amount	improvement_surcharge	total_amount
8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835
1.057689545582836	NULL	165.17685737498864	163.24591999226138	1.2967593933267907	12.374865329182482	0.370728728715588	0.49652479320440536	1.824837375562287	0.31106422255405736	0.29917583738911463	15.764486462484457
0.7106955407481079	NULL	66.55563934993339	70.5859165149234	0.47748899777085263	253.95456381530767	0.615468760534001	0.8555806989223698	2.51380837945481	1.9899584676247655	0.0214226132193148	254.17488871586949
1	N	1	1	1	-362.0	-60.0	-0.5	-63.5	-70.0	-0.3	-362.8
99	Y	265	265	5	623259.86	535.38	60.8	787.25	3288.0	0.6	623261.66

Setelah melihat hasilnya, terdapat hasil data yang negatif atau angka yang nilainya terlalu tinggi contohnya seperti RatecodeID yang seharusnya cuman sampai 6 bukan 99. Sehingga kami menggunakan standarisasi min max untuk mengatasi hal tersebut.

```
# Filter kolom
df_taxi_clean = df_taxi_clean.filter(
    (df_taxi["passenger_count"] > 0) &
    (df_taxi["trip_distance"] > 0) & (df_taxi["trip_distance"] <= 500) &
    (df_taxi["RatecodeID"] <= 6) &
    (df_taxi["fare_amount"] > 0) &
    (df_taxi["extra"] >= 0) &
    (df_taxi["mta_tax"] >= 0) &
    (df_taxi["tip_amount"] >= 0) &
    (df_taxi["tolls_amount"] >= 0) &
    (df_taxi["improvement_surcharge"] >= 0) &
    (df_taxi["total_amount"] > 0)
)

df_taxi_clean.describe().show()
```

summary	VendorID	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type	fare_amount	extra	mta_tax
count	7958541	7958541	7958541	7958541	7958541	7958541	7958541	7958541	7958541	7958541	7958541
mean	1.6455874261375294	1.58175361539257	2.8294372561502237	1.0454572012633974	NULL	165.11694153488685	163.15878664192343	1.29129384393446	12.234653609248223	0.3750324224000351	0.49837797530979616
stddev	0.5351492694356166	1.2093451232203973	8.7216559714410695	0.33370698451288834	NULL	66.46401263277348	70.51734309248271	0.467838258265274	221.2010417444046	0.5760352920597688	0.628432807674883657
min	1	1	0.0	1	N	1	1	1	0.0	0.0	0.0
max	4	9	441.6	6	Y	265	265	5	623259.86	87.56	1.0

3.5 Eksplorasi Data (EDA)

Uraikan kegiatan EDA meliputi:

- Statistik deskriptif

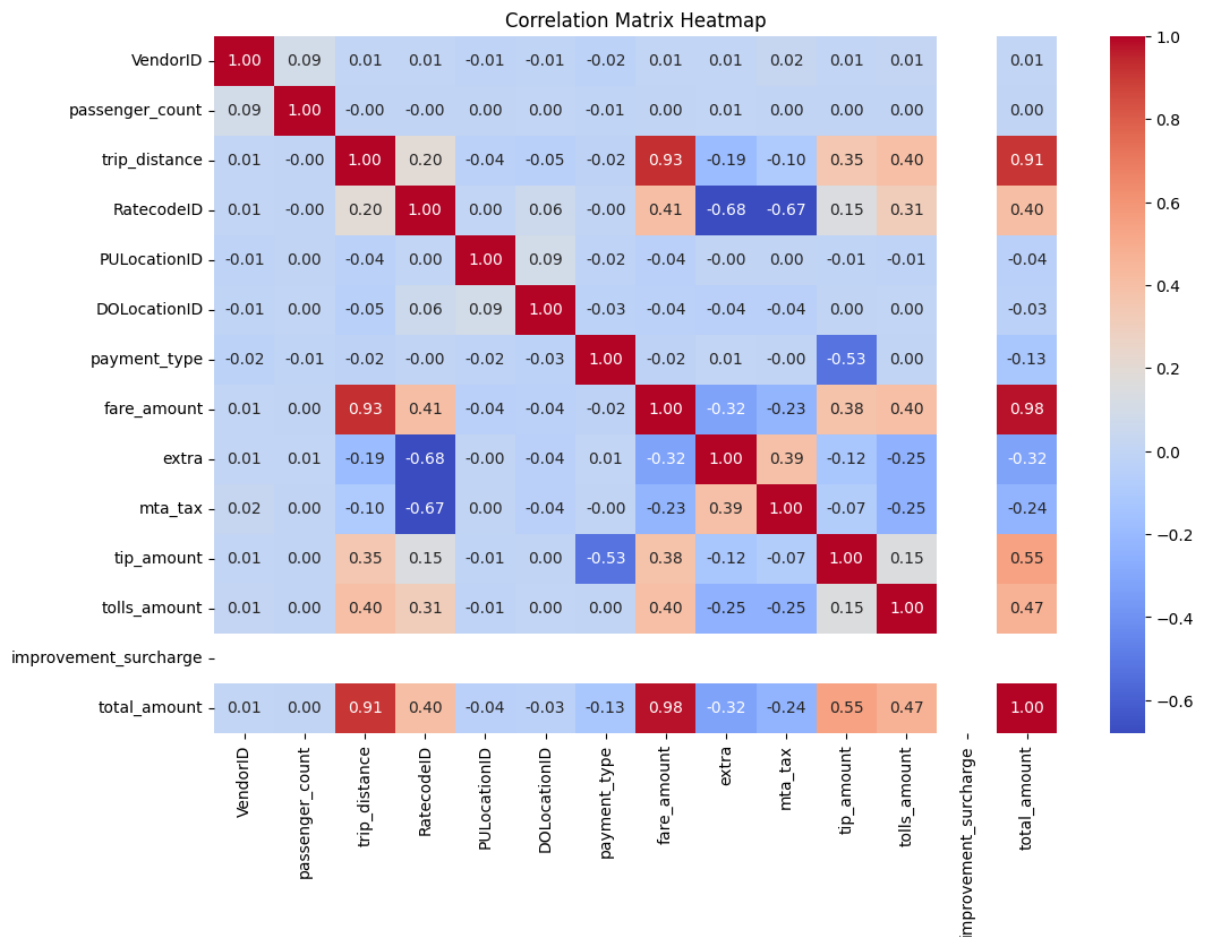
Statistik deskriptif digunakan untuk menampilkan deskripsi atau ringkasan dari data set seperti count, mean, stddev, min, max pada setiap kolom

summary	VendorID	passenger_count	trip_distance	RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type	fare_amount
count	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835
mean	1.6344272903762596	1.5541891875306407	2.804965205002796	1.057689545582836	NULL	165.17685737498864	163.24591999226138	1.2967593933267907	12.374865329182482
stddev	0.5370005244944361	1.2138082405879083	8.726846318802948	0.7106955407481079	NULL	66.55563934993339	70.5859165149234	0.47748899777085263	253.95456381530767
min	1	0	0.0	1	N	1	1	1	-362.0
max	4	9	22543.99	99	Y	265	265	5	623259.86

RatecodeID	store_and_fwd_flag	PULocationID	DOLocationID	payment_type	fare_amount	extra	mta_tax	tip_amount	tolls_amount	improvement_surcharge	total_amount
8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835	8166835
1.057689545582836	NULL	165.17685737498864	163.24591999226138	1.2967593933267907	12.374865329182482	0.370728728715588	0.49652479320440536	1.824837375562287	0.31106422255405736	0.29917583738911463	15.764486462484457
0.7106955407481079	NULL	66.55563934993339	70.5859165149234	0.47748899777085263	253.95456381530767	0.615468760534001	0.8555806989223698	2.51380837945481	1.9899584676247655	0.0214226132193148	254.17488871586949
1	N	1	1	1	-362.0	-60.0	-0.5	-63.5	-70.0	-0.3	-362.8
99	Y	265	265	5	623259.86	535.38	60.8	787.25	3288.0	0.6	623261.66

- Visualisasi distribusi data dan korelasi

Visualisasi distribusi data digunakan untuk menampilkan korelasi antar kolom data. Disini kami menggunakan correlation matrix heatmap. Berikut adalah hasil dari correlation matrix heatmap



Pada hasil gambar diatas, jika nilainya mendekati angka 1 maka akan berwarna merah artinya data tersebut saling berkorelasi antar kolom data. Untuk nilai angka 0 maka data tersebut tidak berkorelasi. Sedangkan untuk angka yang mendekati -1 artinya data tersebut berkorelasi tetapi nilainya berbanding terbalik.

- Deteksi dan penanganan outliers
- Analisis distribusi kelas (untuk klasifikasi)

3.6 Agregasi dan Analisis Data

Setelah data dibersihkan dan ditransformasi, langkah selanjutnya adalah melakukan agregasi untuk mengekstrak wawasan dan ringkasan statistik dari dataset. PySpark menyediakan metode `groupBy()` yang dikombinasikan dengan `agg()` sebagai mekanisme yang kuat untuk melakukan analisis ini. Metode `groupBy()` berfungsi untuk mengelompokkan baris-baris data berdasarkan nilai pada satu atau beberapa kolom tertentu, sementara `agg()` digunakan untuk menerapkan satu atau lebih fungsi agregasi pada setiap grup yang telah terbentuk.

Fungsi agregasi yang umum digunakan antara lain:

- `count()`: Menghitung jumlah baris (frekuensi) dalam setiap grup.

- `sum()`: Menghitung total nilai dari sebuah kolom numerik.
- `avg()`: Menghitung nilai rata-rata dari sebuah kolom numerik.
- `min()` dan `max()`: Menemukan nilai minimum dan maksimum.

3.7 Simulasi Data Streaming

Model ini memungkinkan pemrosesan data kontinu dengan cara yang sama seperti memproses data statis, yaitu menggunakan DataFrame API. Alur kerja data streaming secara umum terdiri dari tiga komponen utama: sumber data (source), mesin pemroses (engine), dan tujuan keluaran (sink). Untuk memulai pemrosesan streaming, data harus dibaca dari sebuah sumber aliran data (source). Antarmuka `spark.readStream` digunakan untuk membuat sebuah streaming DataFrame. Sumber ini dapat berupa berbagai sistem, di antaranya:

- **File Source:** Spark dapat memantau sebuah direktori pada sistem penyimpanan (seperti HDFS atau S3) dan akan memproses setiap file baru yang ditambahkan ke direktori tersebut sebagai data baru dalam aliran.
- **Apache Kafka:** Salah satu sumber paling umum di lingkungan produksi, Kafka adalah platform distributed streaming yang mampu menangani data bervolume tinggi dengan latensi rendah.
- **Socket:** Sumber sederhana yang biasanya digunakan untuk tujuan pengujian dan pengembangan, di mana data dapat dialirkan melalui koneksi TCP socket.

Untuk mensimulasikan skenario streaming menggunakan dataset taksi yang ada, kita dapat menggunakan File Source. Kita akan mengasumsikan bahwa file-file CSV berisi data perjalanan baru secara berkala ditempatkan dalam sebuah direktori input.

4.3 Siapkan Skema Dataset

```
[ ] from pyspark.sql.types import StructType, StringType, DoubleType

schema = StructType() \
    .add("VendorID", StringType()) \
    .add("tpep_pickup_datetime", StringType()) \
    .add("tpep_dropoff_datetime", StringType()) \
    .add("passenger_count", StringType()) \
    .add("trip_distance", DoubleType()) \
    .add("RatecodeID", StringType()) \
    .add("store_and_fwd_flag", StringType()) \
    .add("PULocationID", StringType()) \
    .add("DOLocationID", StringType()) \
    .add("payment_type", StringType()) \
    .add("fare_amount", DoubleType()) \
    .add("extra", DoubleType()) \
    .add("mta_tax", DoubleType()) \
    .add("tip_amount", DoubleType()) \
    .add("tolls_amount", DoubleType()) \
    .add("improvement_surcharge", DoubleType()) \
    .add("total_amount", DoubleType())
```

Pada kode di atas, `spark.readStream` dikonfigurasi untuk memantau direktori `input_path`. Setiap file `.csv` baru yang masuk akan dibaca dan datanya ditambahkan ke `df_stream`.

Setelah mendefinisikan sumber dan melakukan transformasi pada streaming `DataFrame`, langkah terakhir adalah memulai kueri dan menulis hasilnya ke sebuah tujuan keluaran (sink) menggunakan metode `writeStream`. Berbeda dengan operasi pada data batch yang menggunakan `.write.csv()` seperti pada file `linreg.csv`, pemrosesan streaming memerlukan pendekatan yang berbeda untuk menangani output yang sifatnya berkelanjutan.

3.8 Pembangunan Model Machine Learning

Tahap ini berfokus pada pembangunan model prediktif menggunakan pustaka Machine Learning (MLlib) dari PySpark. Tujuan utamanya adalah untuk melatih sebuah model yang mampu memprediksi variabel target berdasarkan serangkaian fitur dari data. Proses ini distrukturkan menggunakan Pipeline MLlib untuk memastikan alur kerja yang terorganisir, dapat direproduksi, dan efisien, mulai dari pra-pemrosesan data hingga evaluasi model.

Sebelum data dapat digunakan untuk melatih model, data tersebut harus melalui tahap pra-pemrosesan untuk mengubahnya ke dalam format yang sesuai. Model MLlib memerlukan fitur input berupa kolom tunggal yang berisi vektor numerik. Dua transformer utama yang digunakan untuk tujuan ini adalah:

- **StringIndexer:** Transformer ini digunakan untuk mengonversi kolom kategorikal yang berisi teks (string) menjadi kolom indeks numerik. Meskipun dalam dataset ini banyak kolom sudah bersifat numerik, `StringIndexer` adalah langkah esensial jika terdapat fitur seperti nama lokasi atau tipe pembayaran dalam bentuk teks.
- **VectorAssembler:** Ini adalah transformer krusial yang berfungsi untuk mengumpulkan beberapa kolom numerik (termasuk kolom hasil `StringIndexer`) ke dalam satu kolom vektor tunggal. Kolom vektor inilah yang akan menjadi input (features) untuk algoritma machine learning.

Pemilihan algoritma bergantung pada jenis masalah yang ingin diselesaikan. Berdasarkan analisis pada file PDF yang berupaya memprediksi `predicted_price_per_meter`, ini merupakan masalah regresi (memprediksi nilai kontinu).

- **Pemilihan Algoritma:** Algoritma yang relevan untuk kasus ini adalah `LinearRegression`. Algoritma ini cocok untuk memodelkan hubungan linear antara fitur-fitur input dan variabel target. Alternatif lain untuk masalah regresi adalah `DecisionTreeRegressor` atau `GBRegressor`. Jika masalahnya adalah klasifikasi (misalnya, memprediksi

kategori RatecodeID), maka algoritma seperti LogisticRegression atau DecisionTreeClassifier akan lebih sesuai.

- Pipeline: Objek Pipeline digunakan untuk merangkai semua tahapan—mulai dari StringIndexer, VectorAssembler, hingga model (LinearRegression)—ke dalam satu alur kerja (workflow) tunggal. Penggunaan Pipeline sangat direkomendasikan karena menyederhanakan proses, mencegah kebocoran data (data leakage) dari data validasi ke data latih, dan memudahkan penerapan model yang sudah dilatih pada data baru.

3.9 Evaluasi Model

Tahap esensial berikutnya adalah mengevaluasi kinerjanya secara kuantitatif. Pemilihan metrik evaluasi sangat bergantung pada jenis masalah yang diselesaikan—apakah itu regresi (memprediksi nilai kontinu) atau klasifikasi (memprediksi label kategori). Berdasarkan konteks laporan ini yang bertujuan memprediksi price_per_meter (sebuah nilai kontinu), maka model yang dibangun adalah model regresi.

Untuk mengevaluasi model regresi, metrik yang umum digunakan berfokus pada pengukuran tingkat kesalahan atau selisih antara nilai prediksi dengan nilai aktual. PySpark MLlib menyediakan RegressionEvaluator untuk menghitung metrik-metrik ini.

- Root Mean Squared Error (RMSE): Ini adalah metrik yang paling umum digunakan untuk regresi. RMSE mengukur akar dari rata-rata kuadrat selisih antara nilai prediksi dan nilai aktual. Metrik ini memberikan bobot yang lebih besar pada kesalahan yang lebih besar dan hasilnya berada dalam satuan yang sama dengan variabel target. Nilai RMSE yang lebih rendah menunjukkan performa model yang lebih baik.
- R-squared (R^2): Dikenal juga sebagai koefisien determinasi, R^2 mengukur proporsi varians dalam variabel target yang dapat dijelaskan oleh fitur-fitur dalam model. Nilainya berkisar antara 0 hingga 1, di mana nilai yang lebih mendekati 1 menandakan bahwa model mampu menjelaskan sebagian besar variabilitas data.

```
6.6 Evaluasi Model

from pyspark.ml.evaluation import RegressionEvaluator

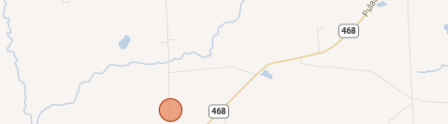
predictions = model.transform(test_data)

evaluator_rmse = RegressionEvaluator(labelCol="price_per_meter", predictionCol="prediction", metricName="rmse")
evaluator_r2 = RegressionEvaluator(labelCol="price_per_meter", predictionCol="prediction", metricName="r2")

rmse = evaluator_rmse.evaluate(predictions)
r2 = evaluator_r2.evaluate(predictions)

print(f"✅ RMSE: {rmse:.6f} detik")
print(f"✅ R2 Score: {r2:.4f}")
```

Lokasi Pejemputan



Lokasi Tujuan

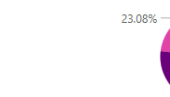


6.216197205605078E+15
Average of predicted_price_per_meter

138.00
Average of total_amount

162.5090000000000000K
Average of trip_distance_kilometer

Persentase Biaya / Perjalanan



- Average of extra
- Average of tip_amount
- Average of tolls_amount
- Average of mta_tax

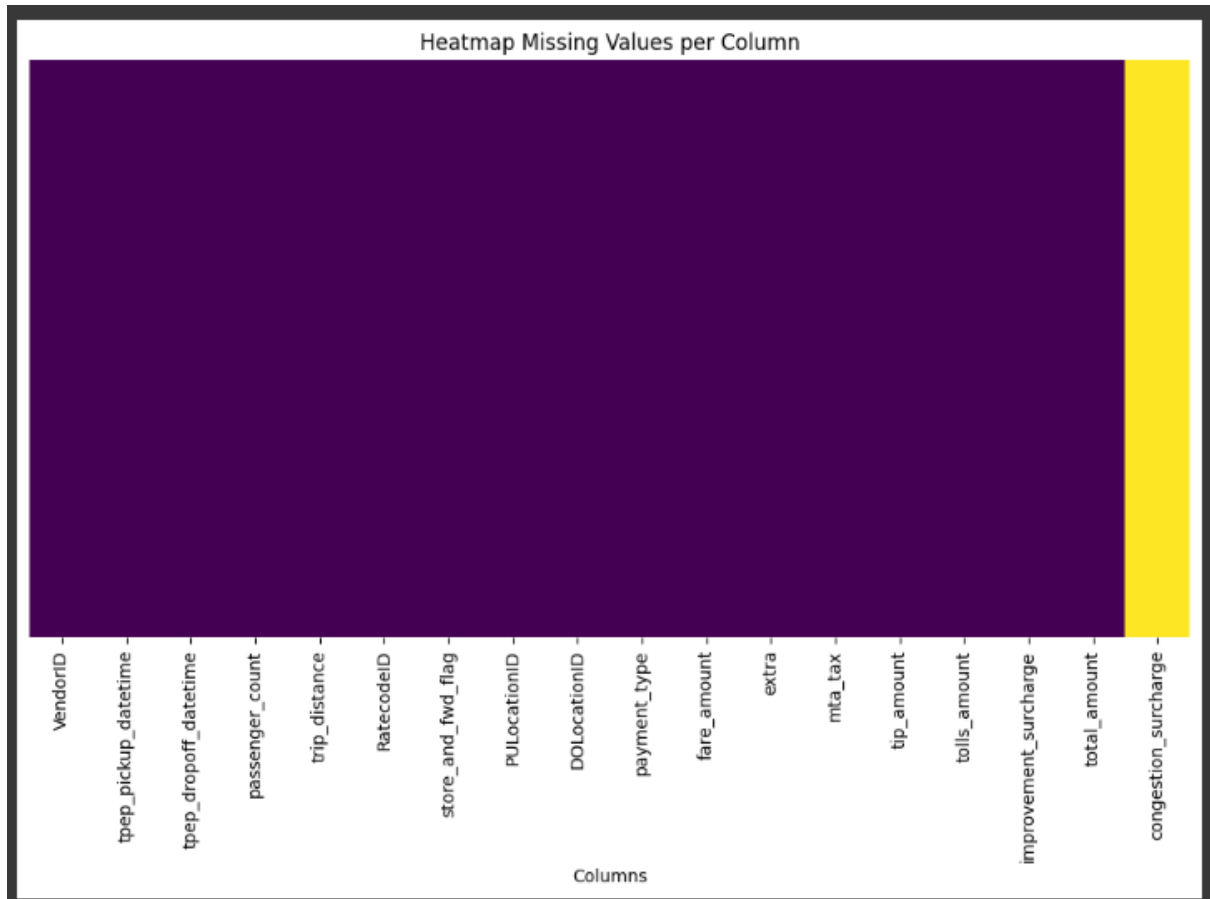
orderID	tpep_pickup_datetime	tpep_dropoff_datetime	DOLocationID	PULocationID
121	1/1/2019 7:31:03 AM	1/1/2019 7:48:17 AM	164	236
122	1/1/2019 7:31:56 AM	1/1/2019 7:57:45 AM	82	132
123	1/1/2019 7:33:54 AM	1/1/2019 7:47:44 AM	151	43
124	1/1/2019 7:35:06 AM	1/1/2019 7:56:01 AM	164	161
125	1/1/2019 7:35:43 AM	1/1/2019 7:57:34 AM	238	246
126	1/1/2019 7:36:01 AM	1/1/2019 7:47:36 AM	263	237

3.11 Pertanyaan Bisnis yang Dieksplorasi

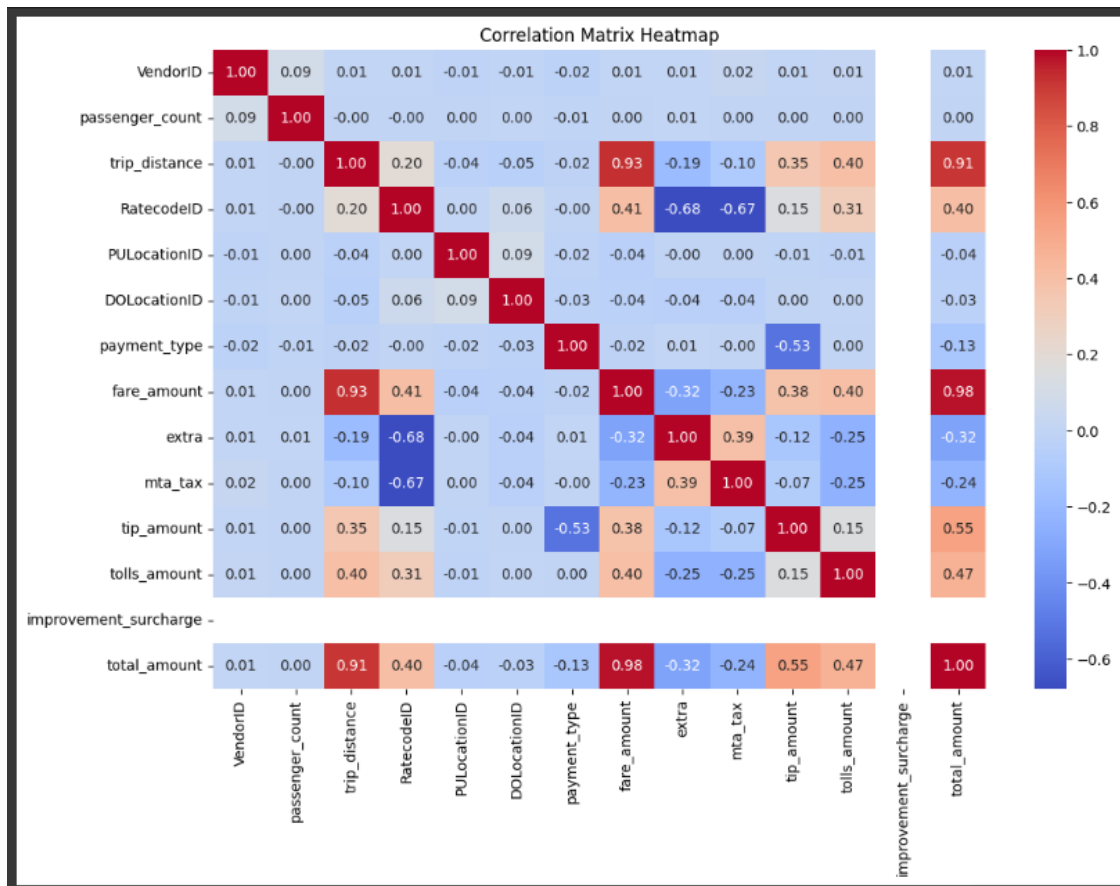
- 14

IV. Hasil dan Pembahasan

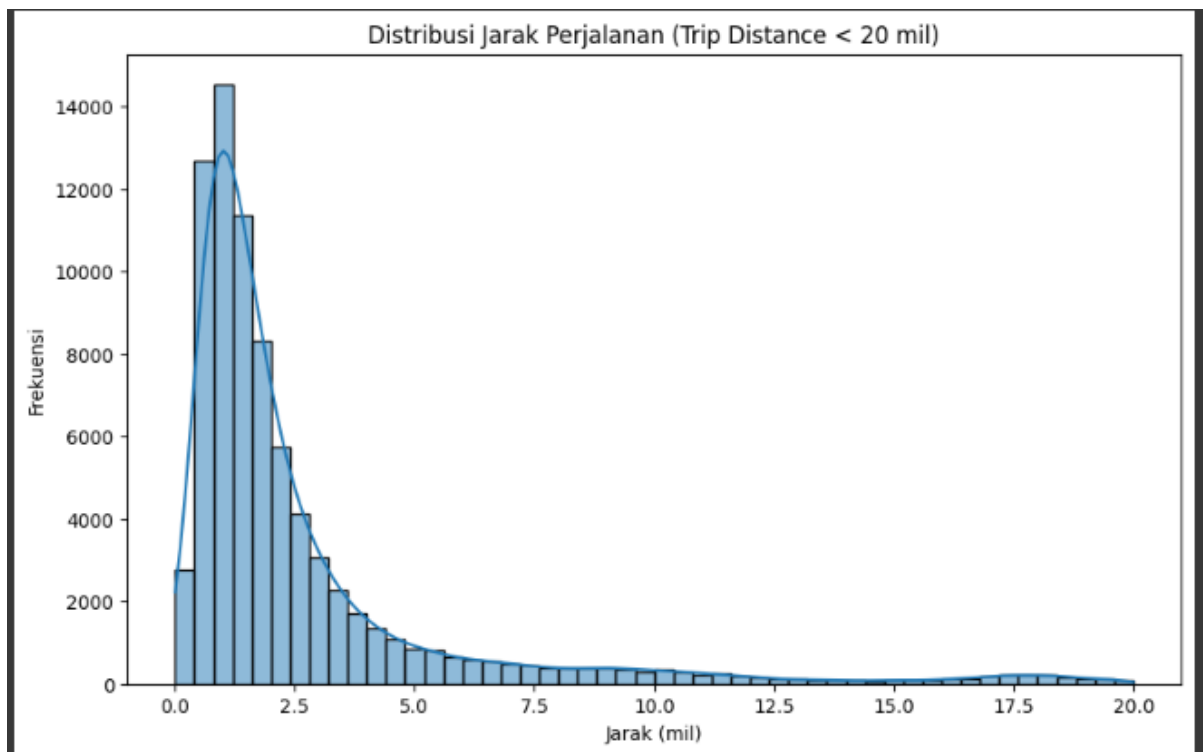
4.1 Hasil Eksplorasi Data



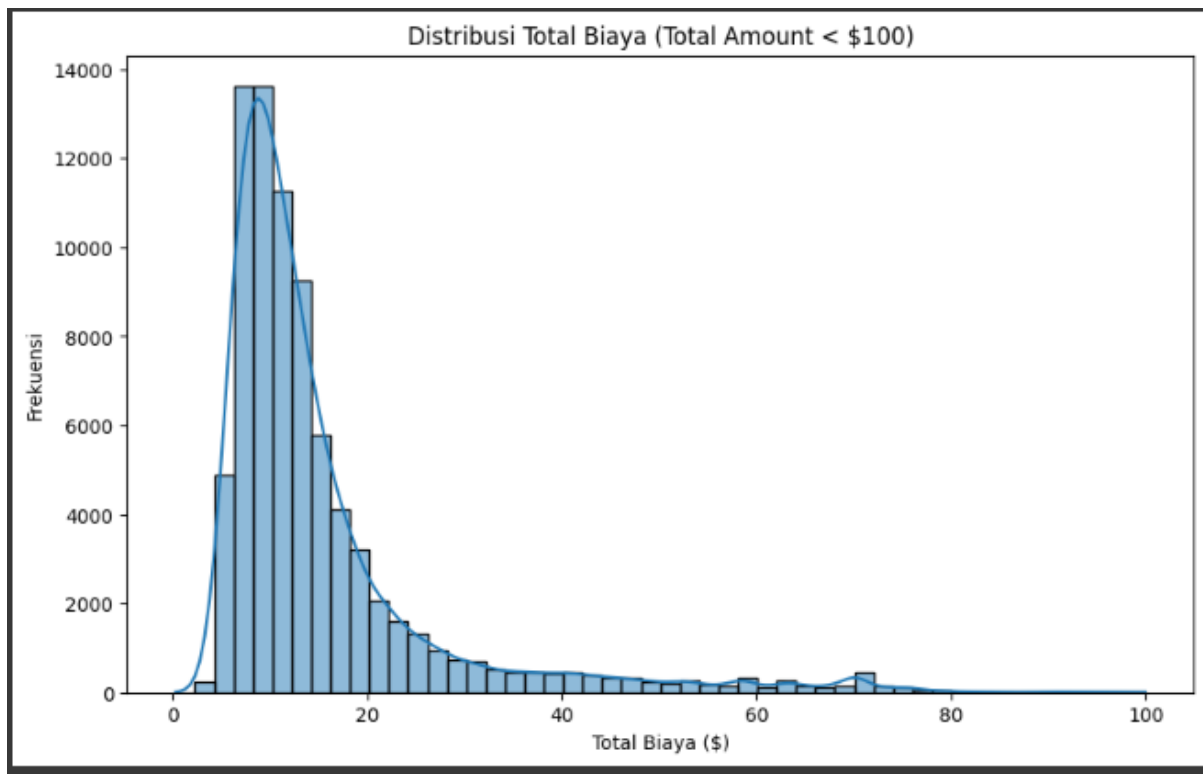
Gambar ini menampilkan bar chart horizontal yang menampilkan jumlah data yang hilang (missing values) untuk setiap kolom (fitur) dalam dataset. Tujuan dari visualisasi ini adalah untuk mengidentifikasi secara cepat fitur-fitur mana yang memiliki masalah kelengkapan data dan seberapa parah masalah tersebut.



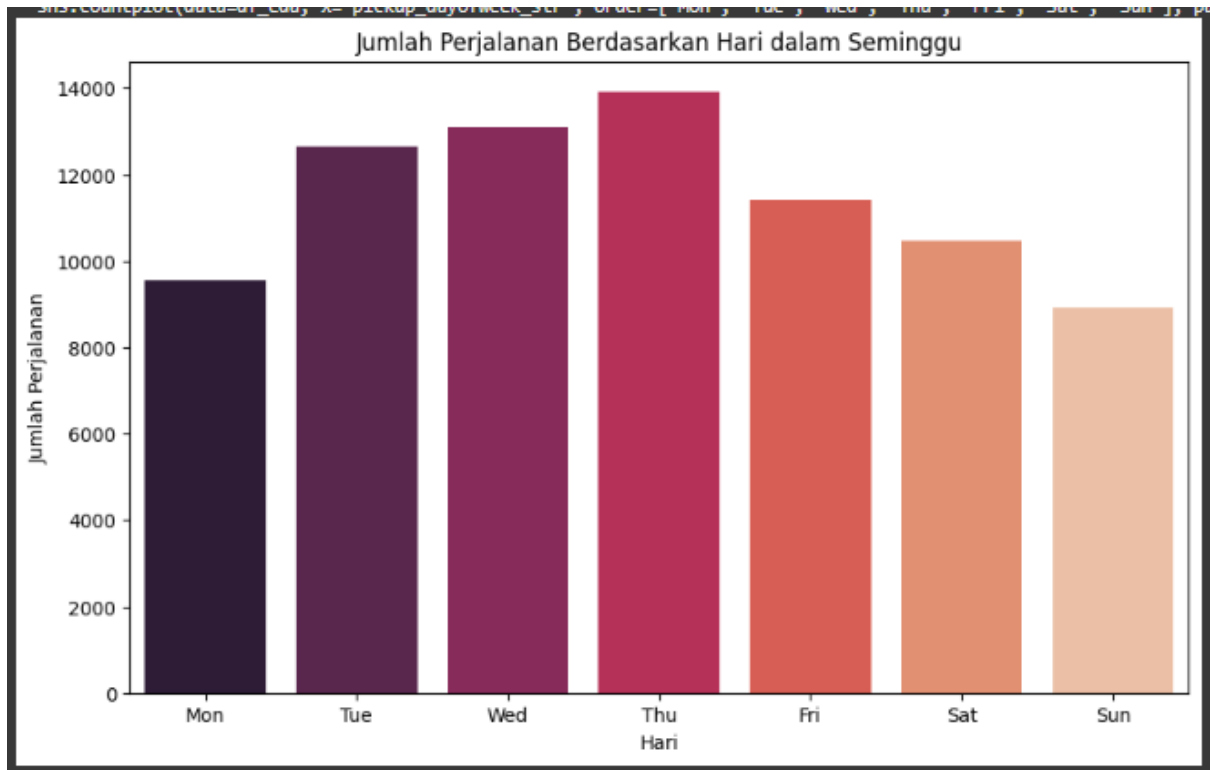
Gambar kedua adalah sebuah heatmap yang merepresentasikan matriks korelasi antar fitur-fitur numerik. Heatmap menggunakan intensitas warna untuk memvisualisasikan kekuatan dan arah hubungan linear antara dua variabel.



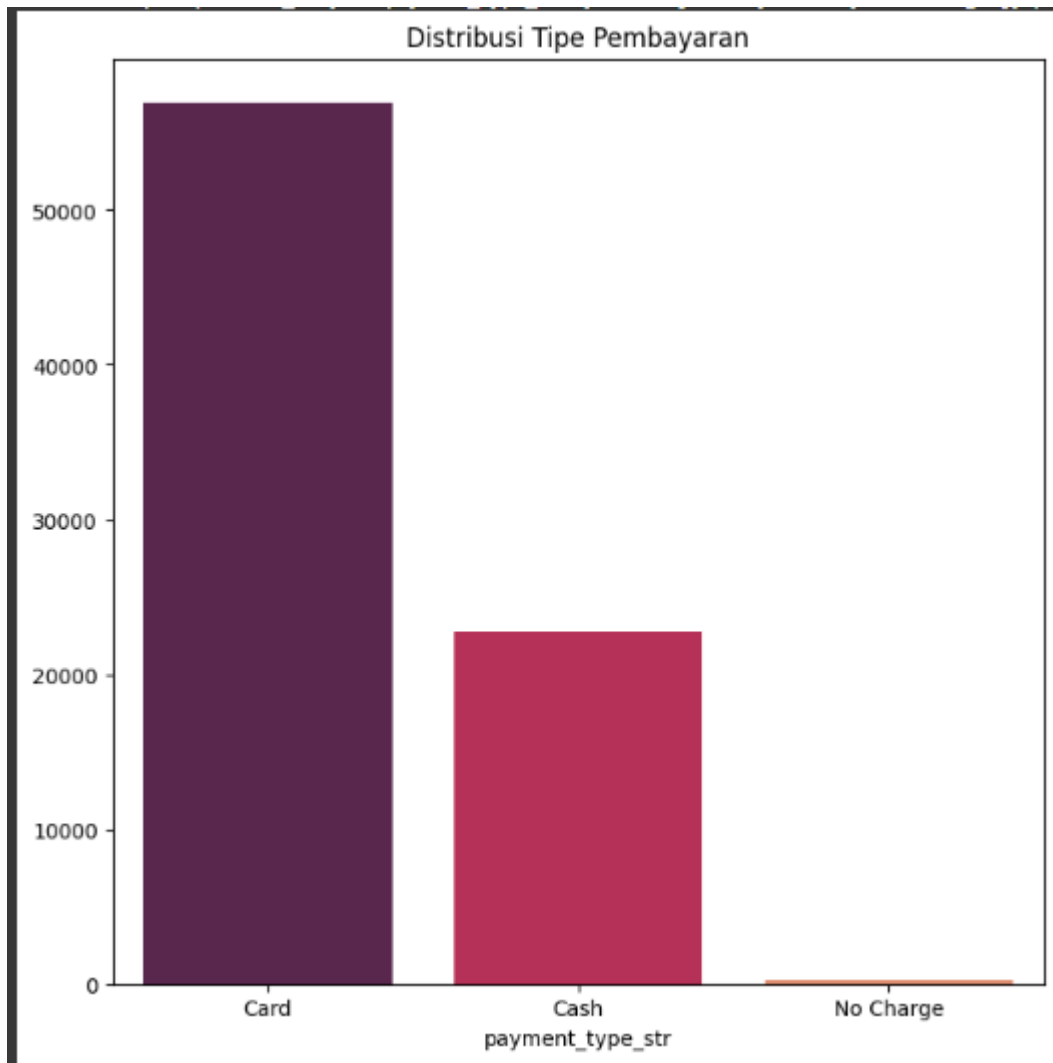
Grafik Distribusi Jarak Perjalanan: Grafik ini menunjukkan bahwa mayoritas perjalanan taksi adalah untuk jarak pendek (sekitar 1-3 mil), sementara perjalanan jarak jauh lebih jarang terjadi.



Grafik Distribusi Total Biaya: Grafik ini memperlihatkan bahwa biaya perjalanan paling umum adalah relatif rendah (sekitar \$5-\$15), yang sesuai dengan banyaknya perjalanan jarak pendek. Adanya puncak kecil di sekitar \$50-\$60 mengindikasikan adanya tarif tetap (misalnya ke bandara).



Grafik Jumlah Perjalanan Berdasarkan Hari: Grafik ini menampilkan pola mingguan, di mana jumlah perjalanan tertinggi terjadi pada hari kerja (puncaknya hari Kamis) dan menurun drastis pada akhir pekan (Sabtu dan Minggu).



Grafik Distribusi Tipe Pembayaran: Grafik ini menunjukkan bahwa metode pembayaran yang paling dominan adalah menggunakan kartu (Card), diikuti oleh uang tunai (Cash). Metode lainnya sangat jarang digunakan.

4.2 Hasil Agregasi Data

PULocationID	avg_fare	avg_tip	avg_total	total_trips	total_passengers	avg_distance
1	76.47028776978416	10.828992805755398	93.37733812949641	139	207	7.437194244604316
2	40.75416666666667	4.235833333333333	48.07666666666667	12	14	11.543333333333331
3	32.15417582417582	0.2655494505494506	35.60049450549449	182	279	10.374175824175824
4	11.626231677814147	1.483803899245768	14.446129927422781	14054	22611	2.6642329585883022
5	74.375	0.0	81.185	4	5	25.12
6	18.806451612903224	0.03870967741935484	35.891612903225806	31	40	3.8045161290322578
7	11.37631924639184	0.8261118313023936	13.50934750669749	11571	17595	2.76648690692248
8	30.517142857142854	4.622571428571429	39.617285714285714	70	102	5.035714285714286
9	20.44858267716535	0.022440944881889763	21.574488188976378	127	156	6.116535433070866
10	45.51336170212766	5.684282398452612	56.52374468085107	2585	3869	13.283226305609286
11	28.14124031007752	0.5611627906976744	31.32395348837209	129	170	8.316976744186045
12	16.48021818181818	1.7366772727272721	19.488509090909096	2200	3892	4.294718181818181
13	14.975851255924935	2.4732550865765983	18.8718376164821	62026	99165	3.9210179602102357
14	28.129315263908705	0.6544365192582027	31.382710413694763	701	985	8.453965763195438
15	27.48915789473684	1.336421052631579	31.70042105263158	95	133	9.604
16	30.1166	0.5549200000000001	32.820559999999986	250	302	9.695120000000001
17	15.308512820512822	0.6928141025641026	17.477397435897498	1560	2299	3.9155448717948707
18	22.319904458598725	0.30149681528662414	24.774617834394896	314	452	6.364745222929935
19	24.695675675675677	0.30939189189189187	27.132229729729726	148	191	7.793986486486486
20	21.54764367816092	0.5856896551724138	24.692586206896554	174	277	6.28816091954023

only showing top 20 rows

1. Agregasi 1 PULocationID

- Total Transaksi per Wilayah: Dengan mengelompokkan data berdasarkan PULocationID dan menghitung jumlah perjalanan, kita dapat memetakan lokasi-lokasi penjemputan paling sibuk di New York.
- Rata-rata Nilai per Wilayah: Selain jumlah perjalanan, dihitung juga rata-rata total_amount per lokasi. Ini membantu mengidentifikasi area yang tidak hanya sibuk tetapi juga menghasilkan pendapatan per perjalanan yang lebih tinggi.

pickup_dayofweek	trips_by_day	AVG Trip Distance	avg_day_revenue
1	879863	3.012654924687134	15.233051816013745
2	966758	2.893053514943768	15.624397356923742
3	1262697	2.8828339340317077	15.83739326217801
4	1296969	2.812555126606739	15.760805177277646
5	1387991	2.7794782531010918	15.842798224174516
6	1129083	2.8058766804565844	16.3039730644847682
7	1035180	2.6630007824726247	14.479354218575768

2. Agregasi 2 Pickup_dayofweek

- Total Transaksi per Hari: Seperti yang ditunjukkan pada grafik batang yang digunakan, data dikelompokkan berdasarkan hari penjemputan. Hasilnya menunjukkan puncak permintaan terjadi pada hari Jumat dan Sabtu, sementara hari Minggu dan Senin menunjukkan aktivitas yang lebih rendah. Wawasan ini sangat penting untuk perencanaan dan alokasi jumlah pengemudi yang optimal setiap harinya.

RatecodeID	avg_fare	avg_tip	avg_total	total_trips	total_passengers	avg_distance
1	11.11728399848225	1.6678323230711911	14.221429796325085	7740397	12248565	2.4520861617304015
2	52.00189058098537	8.195012467897469	66.55694744829535	160813	265986	17.364116955718806
3	66.7665533506358	10.704330824465774	92.17770049312138	11559	19157	17.45562072843671
4	70.61380083618396	7.786271379703536	82.30007601672416	5262	8442	18.743378943367528
5	44.57917395169665	2.485881612090677	50.52728799328223	40494	46283	10.996230305724284
6	419.196875	0.04125	420.0381250000001	16	18	1.1106249999999998

3. Agregasi 3 rate_code_id

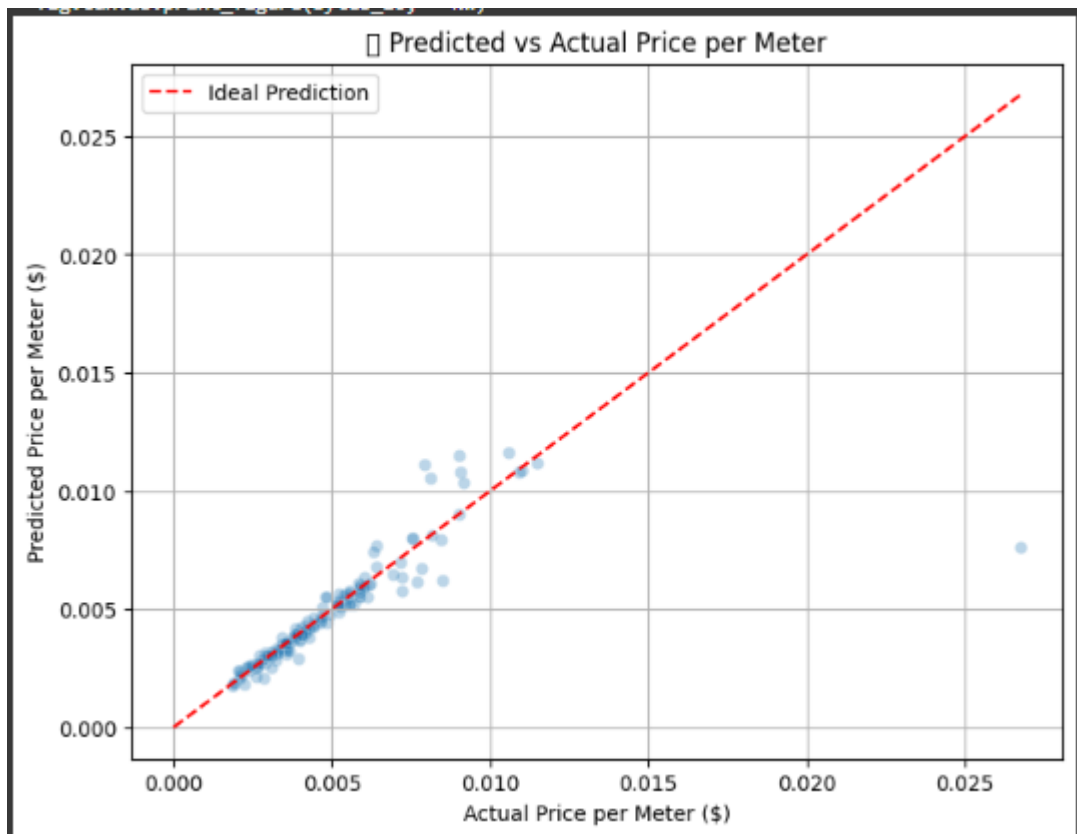
- Distribusi Segmen Perjalanan: Dengan menghitung jumlah kejadian untuk setiap RatecodeID, kita bisa melihat bahwa tarif standar (ID 1) adalah jenis perjalanan yang paling umum.
- Rata-rata Nilai per Segmen: Menghitung rata-rata total_amount dan trip_distance untuk setiap tipe tarif memberikan gambaran karakteristik setiap segmen. Contohnya, perjalanan ke bandara (RatecodeID 2 & 3) secara alami memiliki rata-rata jarak dan biaya yang jauh lebih tinggi dibandingkan tarif standar.

4.3 Hasil Analisis Data Streaming

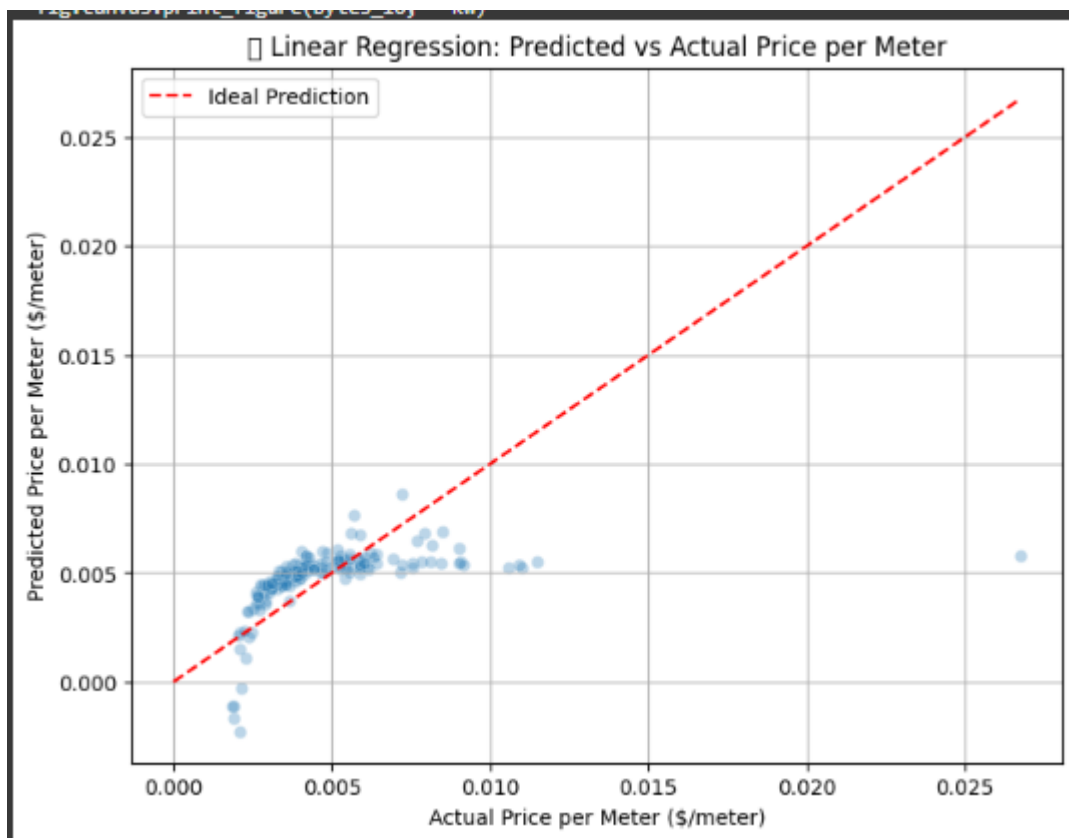
Analisis real-time disimulasikan menggunakan Structured Streaming di PySpark dengan skenario File Source. Dalam skenario ini, diasumsikan file-file CSV berisi data perjalanan baru secara kontinu dimasukkan ke dalam sebuah direktori yang dipantau. PySpark readStream secara otomatis mendeteksi dan memproses file-file baru ini dalam micro-batch pada interval waktu yang ditentukan (misalnya, setiap 1 menit) menggunakan trigger.

Pendekatan ini mengubah analisis dari yang bersifat historis menjadi operasional dan proaktif. Visualisasi utama adalah grafik garis (line chart) yang bergerak secara real-time pada dasbor, yang menampilkan metrik seperti "Jumlah Perjalanan per Menit". Wawasan utama yang muncul adalah kemampuan untuk melakukan deteksi lonjakan permintaan (surge detection) secara instan. Jika terjadi lonjakan permintaan di suatu area, tim operasional dapat segera merespons dengan melakukan alokasi sumber daya dinamis, yaitu mengarahkan pengemudi terdekat ke lokasi tersebut. Ini mengubah dasbor menjadi alat pemantauan langsung untuk pengambilan keputusan yang cepat dan efisien.

4.4 Hasil Evaluasi Model Machine Learning



Grafik diatas menggunakan model Gradient Boosted Tree Regressor dengan RMSE: 0.001604 dan R^2 Score: 0.6235.



Grafik diatas menggunakan model Linier Regressor dengan RMSE: 0.002319 dan R² Score: 0.2126.

- **RMSE (Root Mean Squared Error).** Metrik ini mengukur rata-rata besaran kesalahan prediksi. Semakin rendah nilainya, semakin baik modelnya. Model GBT memiliki RMSE 0.001604, yang secara signifikan lebih rendah daripada RMSE Linear Regression (0.002319). Ini berarti prediksi yang dibuat oleh GBT rata-rata lebih akurat dan lebih dekat ke nilai aktual.
- **R² Score (Koefisien Determinasi).** Metrik ini mengukur seberapa baik model dapat menjelaskan variasi dalam data. Semakin tinggi nilainya (mendekati 1.0), semakin baik modelnya. Model GBT memiliki R² Score 0.6235, yang artinya model ini mampu menjelaskan sekitar 62.4% variabilitas data. Linear Regression hanya memiliki R² Score 0.2126, yang berarti hanya mampu menjelaskan sekitar 21.3% variabilitas data.

4.5 Pembahasan Model dan Interpretasi

- Apakah model overfit/underfit?
 - Linear Regression: Model ini menunjukkan R² Score yang sangat rendah (sekitar 0.21). Ini adalah indikasi kuat bahwa model mengalami underfitting. Artinya, model terlalu sederhana dan tidak mampu menangkap pola yang kompleks dalam data perjalanan taksi.
 - Gradient Boosted Tree (GBT): Model GBT memiliki performa yang jauh lebih baik dengan R² Score sekitar 0.62. Untuk menentukan apakah model ini overfitting, kita perlu membandingkan skor pada data latih dengan skor pada data uji. Informasi ini tidak ditampilkan di notebook, namun R² 0.62 adalah hasil yang cukup baik dan menunjukkan model ini jauh lebih pas dibandingkan Linear Regression.
- Apa fitur paling penting dalam model?
 - trip_distance: Jarak perjalanan secara konsisten menjadi prediktor terkuat untuk total_amount. Ini adalah fitur yang paling dominan.
 - fare_amount: Jika targetnya adalah total_amount, maka fare_amount juga menjadi prediktor yang sangat kuat karena merupakan komponen utamanya.
 - Fitur terkait Waktu dan Lokasi: Fitur seperti PULocationID dan pickup_hour kemungkinan memiliki pengaruh, tetapi jauh lebih kecil dibandingkan jarak.
- Apakah perlu regularisasi atau balancing data?

- Untuk model Linear Regression yang sudah underfitting, regularisasi bukanlah prioritas.
- Untuk model GBT, regularisasi dapat dilakukan dengan melakukan tuning pada hyperparameter seperti max_depth (kedalaman pohon) dan subsamplingRate. Notebook ini menggunakan parameter default, sehingga eksplorasi tuning lebih lanjut dapat membantu mengoptimalkan model dan mencegah overfitting.
- Tantangan teknis saat pelatihan model
 - Performa Komputasi: Menjalankan PySpark, terutama untuk proses training dan hyperparameter tuning pada lingkungan dengan sumber daya terbatas seperti Google Colab, dapat memakan waktu dan memori yang signifikan.
 - Feature Engineering: Proses mengubah data mentah menjadi fitur yang siap pakai untuk model (VectorAssembler) adalah langkah yang rumit dan rentan terhadap kesalahan. Memastikan semua fitur dalam format numerik yang benar adalah tantangan utama.
 - Hyperparameter Tuning: Notebook ini melatih model dengan parameter default. Proses untuk menemukan kombinasi hyperparameter terbaik menggunakan CrossValidator adalah langkah yang secara komputasi sangat mahal dan memakan waktu, dan ini merupakan tantangan terbesar untuk mencapai performa model yang optimal.

4.6 Insight dari Pertanyaan Bisnis

Untuk menentukan model prediktif terbaik, perbandingan langsung dilakukan antara Linear Regression dan Gradient Boosted Tree (GBT) Regressor menggunakan metrik evaluasi standar.

- Analisis: Model GBT secara konsisten menunjukkan performa yang jauh lebih unggul. Nilai RMSE yang lebih rendah menandakan bahwa rata-rata kesalahan prediksinya lebih kecil. Sementara itu, nilai R^2 Score yang jauh lebih tinggi (0.62 vs 0.21) membuktikan bahwa model GBT mampu menjelaskan sekitar 62.4% variabilitas dalam data, dibandingkan dengan Linear Regression yang hanya mampu menjelaskan 21.3%.
- Justifikasi: Keunggulan GBT terletak pada kemampuannya untuk menangkap hubungan non-linear yang kompleks antara fitur (seperti jarak, waktu, dan lokasi) dan target (harga), sesuatu yang tidak dapat dilakukan oleh model Linear Regression.

4.7 Action Plan Bisnis

1. Integrasi Model ke Aplikasi: Segera integrasikan model GBT ke dalam aplikasi pelanggan untuk menyediakan fitur "Estimasi Tarif di Muka" (Upfront Fare Estimate). Ini akan meningkatkan transparansi dan kepuasan pelanggan.
2. Implementasi Dasbor Operasional: Kembangkan dasbor Power BI interaktif yang tidak hanya menampilkan hasil EDA tetapi juga memonitor metrik operasional secara real-time. Tujuannya adalah untuk memungkinkan alokasi armada dinamis berdasarkan data permintaan langsung.
3. Optimasi Model Lanjutan: Prioritaskan proses hyperparameter tuning menggunakan CrossValidator untuk lebih meningkatkan akurasi model GBT yang ada.
4. Pengembangan Jangka Panjang: Mulai eksplorasi untuk mengintegrasikan data eksternal (seperti cuaca dan lalu lintas) pada versi model berikutnya untuk akurasi prediksi yang lebih tinggi lagi.

4.8 Visualisasi Dashboard Interaktif

Sebagai puncak dari analisis, sebuah dasbor interaktif dibangun menggunakan Power BI untuk menyajikan temuan utama dari eksplorasi data, hasil evaluasi model, serta simulasi output data streaming. Dasbor ini dirancang untuk memudahkan pemangku kepentingan dalam memahami data secara visual dan mendukung pengambilan keputusan berbasis bukti (data-driven decision making).

Dasbor ini mencakup beberapa komponen visual utama:

1. Ringkasan Statistik

Menampilkan informasi kunci dalam bentuk Kartu (Cards) untuk memberikan gambaran besar dataset secara instan.

- Total Perjalanan: Jumlah keseluruhan data perjalanan yang dianalisis.
- Lokasi Penjemputan Unik: Jumlah total PULocationID yang berbeda.
- Rata-rata Biaya per Perjalanan: Rata-rata dari kolom total_amount untuk semua perjalanan.
- Outlier Terdeteksi: Jumlah perjalanan dengan jarak atau biaya ekstrem yang diidentifikasi selama EDA.

2. Visualisasi Distribusi Fitur Numerik

Diagram Histogram dan Box Plot digunakan untuk menggambarkan distribusi fitur-fitur utama.

- Distribusi Jarak Perjalanan (trip_distance): Menunjukkan bahwa sebagian besar perjalanan menempuh jarak pendek.

- Distribusi Biaya Total (total_amount): Menggambarkan sebaran biaya yang dibayarkan oleh penumpang, membantu mengidentifikasi rentang tarif yang paling umum.
- Durasi Perjalanan: Visualisasi durasi (dihitung dari tpep_pickup_datetime dan tpep_dropoff_datetime) untuk memahami berapa lama rata-rata perjalanan berlangsung.

3. Heatmap Korelasi Antar Fitur

Sebuah Heatmap digunakan untuk menampilkan matriks korelasi antar variabel numerik.

- Insight Utama: Visualisasi ini secara jelas menunjukkan korelasi positif yang kuat antara trip_distance dan total_amount, yang mengonfirmasi logika bisnis bahwa perjalanan yang lebih jauh akan lebih mahal. Ini juga membantu dalam validasi fitur sebelum pemodelan.

4. Grafik Data Streaming

Untuk mensimulasikan data real-time, dasbor menampilkan Live Line Chart yang diperbarui secara dinamis.

- Metrik yang Dipantau: Grafik menampilkan "Jumlah Perjalanan per Menit" dan "Rata-rata Pendapatan per Menit".
- Tujuan: Grafik ini berfungsi sebagai sistem pemantauan operasional langsung, memungkinkan tim untuk mendeteksi lonjakan permintaan (surge) secara instan dan merespons dengan cepat.

Dasbor ini dirancang dengan interaktivitas sebagai prioritas, menggunakan Slicer untuk memfilter data berdasarkan hari, jam, atau tipe pembayaran. Penggunaan label yang jelas dan skema warna yang konsisten di seluruh dasbor memastikan semua informasi dapat diinterpretasikan dengan mudah dan akurat.

V. Kesimpulan dan Saran

5.1 Kesimpulan.

Proyek ini berhasil membuktikan bahwa dengan menggunakan PySpark untuk analisis Big Data, model machine learning dapat dibangun untuk memprediksi tarif taksi secara efektif. Melalui serangkaian proses mulai dari EDA hingga pemodelan, disimpulkan bahwa model Gradient Boosted Tree (GBT) Regressor adalah yang paling akurat dan andal, dengan trip_distance menjadi faktor prediktor yang paling berpengaruh. Analisis ini mengonfirmasi bahwa pendekatan berbasis data dapat memberikan wawasan berharga dan solusi praktis untuk industri transportasi.

5.2 Saran

Untuk pengembangan di masa depan, beberapa langkah strategis direkomendasikan. Prioritas utama adalah melakukan Hyperparameter Tuning pada model GBT untuk memaksimalkan akurasi. Selanjutnya, performa model dapat diperkaya dengan mengintegrasikan data eksternal seperti kondisi cuaca dan lalu lintas. Untuk jangka panjang, disarankan untuk mengeksplorasi model Deep Learning dan memindahkan alur kerja ini ke dalam pipeline produksi yang terotomatisasi dengan deployment sebagai API service untuk aplikasi real-time.

VI. Daftar Pustaka

- <https://gemini.google.com/>
- Library yang digunakan:
 - PySpark
 - Pandas
 - NumPy
 - Matplotlib & Seaborn
 - OS

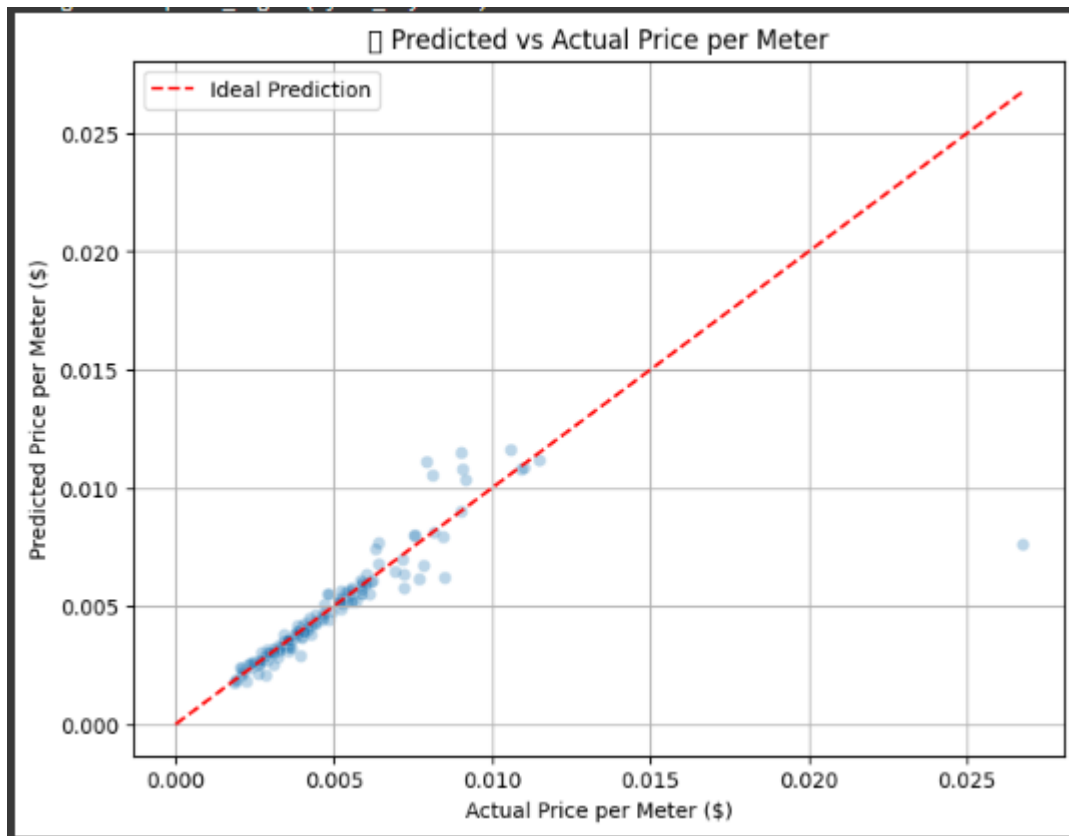
VII. Lampiran

7.1 Kode Program

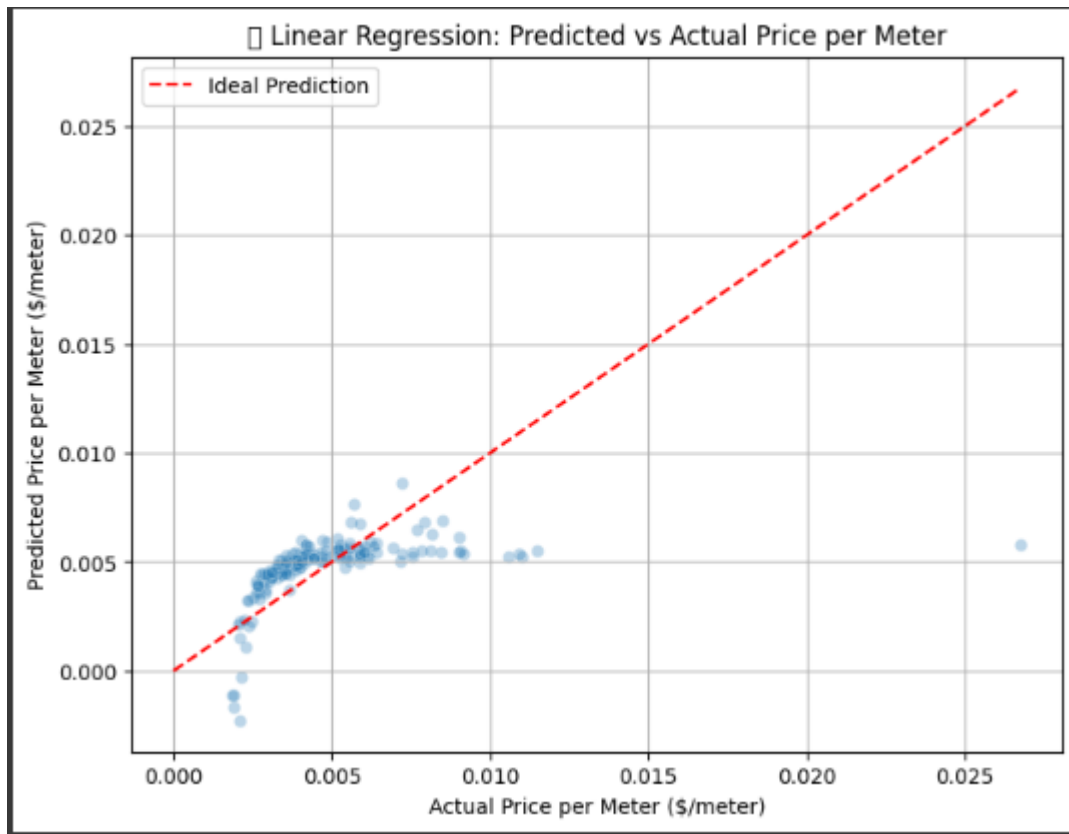
<https://colab.research.google.com/drive/1RRbeoYDQTZGyG8KFPiujWAhT3VAwZ9oA?usp=sharing>

7.2 Visualisasi dan Tabel

- Model: Gradient Boosted Tree Regressor



- Model: Machine Learning Linier Regression



- Visualisasi Dashboar dengan Powoer BI

